

AUTOMATED NETWORK REQUEST MANAGEMENT IN SERVICENOW

A Project Report Submitted for
ServiceNow Certified Application Developer (CAD) Program

Author	Kotha Nikhil
College	MLR Institute of Technology
Email	23r21a05a1@mlrit.ac.in
Program	SmartInternz – ServiceNow Certified Application Developer
Platform	ServiceNow Developer Instance (PDI)

This report documents the design, configuration, and implementation of an automated network service-request management solution built on the ServiceNow platform, covering service catalog design, workflow automation, approvals, and integration considerations.

Table of Contents

1.	Introduction	3
2.	Project Objectives	4
3.	Scope of the Project	5
4.	Key Features	6
5.	System Architecture and Design	7
6.	ServiceNow Developer Environment Setup	8
7.	Project Implementation in ServiceNow	9
8.	Testing and Validation	12
9.	Screenshots of Output	13
10.	Results and Benefits	14
11.	Challenges and Learnings	14
12.	Future Enhancements	15
13.	Conclusion	15
14.	References	15

1. Introduction

Modern enterprises depend heavily on reliable and well-managed network infrastructure to support day-to-day business operations. Employees regularly require new network connections, relocation of existing connections, device provisioning, and troubleshooting support. Traditionally, such requests are handled manually through emails, phone calls, or spreadsheets, which often leads to delays, miscommunication, lost requests, and a lack of visibility into request status.

This project, developed as part of the SmartInternz ServiceNow Certified Application Developer program, addresses these challenges by designing and implementing an Automated Network Request Management system on the ServiceNow platform. The solution makes use of ServiceNow's Service Catalog, Flow Designer, approval workflows, and notification engine to streamline the complete lifecycle of a network request, from submission to closure.

By moving the network request process onto a structured, automated platform, organizations can significantly reduce manual effort, enforce consistent approval processes, maintain an auditable trail of all requests, and provide end users with real-time visibility into the status of their requests.

2. Project Objectives

The primary goal of this project is to design and build an end-to-end automated workflow for managing network service requests within ServiceNow. The specific objectives are as follows:

- Design and implement an automated solution for managing network service requests, from submission through fulfillment and closure.
- Provide a user-friendly self-service portal through which employees can raise network-related requests without needing to contact IT staff directly.
- Automate notifications, task assignments, and approvals so that requests move through the process without manual intervention.
- Enable optional integration with network automation tools or scripts to support standard, repeatable request types such as device provisioning.
- Improve Service Level Agreement (SLA) compliance through structured workflows and automated escalation.
- Increase transparency for end users by providing real-time status updates on their requests.

3. Scope of the Project

The scope of this project covers the design and configuration of a custom Service Catalog item for network requests, the underlying data model used to store request details, the approval and fulfillment workflow built using Flow Designer, and the notification mechanisms that keep requesters and approvers informed throughout the process.

The project is implemented entirely on a ServiceNow Personal Developer Instance (PDI), which provides a fully functional sandbox environment suitable for application development and testing without affecting any production system. While the project demonstrates the core automation pipeline, it is designed to be extensible so that organizations can later connect it to real network automation platforms such as Cisco DNA Center, Ansible, or other orchestration tools.

In-Scope Items

- Custom Service Catalog item ('Network Request') with dynamic variables.
- Custom database table to store network request records.
- Approval relationship configuration using the Sysapprovals table.
- Flow Designer workflow covering trigger, data capture, approval, and notification.
- Email notifications for request submission, approval, and completion.

Out-of-Scope Items

- Direct, production-grade integration with live network hardware or SDN controllers.
- Advanced reporting dashboards beyond the standard ServiceNow reporting tools.
- Mobile application development (the solution uses the standard ServiceNow portal/app).

4. Key Features

The Automated Network Request Management solution incorporates the following key features, each of which contributes to reducing manual effort and improving the overall request handling experience.

Feature	Description
Service Catalog with Dynamic Variables	A custom 'Network Request' catalog item is built in ServiceNow's Service Catalog module. The form uses catalog variables of different types — Choice (new connection / relocation), String (relocated address, device details), and Reference — so the correct fields are presented based on the user's selection. This ensures clean, structured data is captured at the point of submission.
Flow Designer Workflow Automation	The entire request lifecycle is automated using ServiceNow's Flow Designer. The flow is triggered by the creation of a Requested Item (RITM) from the catalog. It then sequentially executes: Get Catalog Variables, Create Record in the custom Network Request table, Ask for Approval, evaluate the outcome using If/Else logic, Send Email notifications, and Update Record — with no manual routing required.
Approval Management via Sysapprovals	Approval functionality is integrated by creating a relationship between the custom Network Request table and ServiceNow's native Sysapprovals table via System Definition → Relationships, then surfaced on the form using Form Designer. Approvers review request details and approve or reject directly from the record; the flow automatically continues based on the decision.
Automated Email Notifications and Record Updates	At each key workflow stage, the Send Email action dispatches contextual notifications to the requester and approver. On approval or rejection, both parties are notified. The Update Record action then sets the final status on both the custom Network Request table record and the originating RITM, maintaining a complete and accurate audit trail throughout.

5. System Architecture and Design

The solution follows a layered architecture typical of ServiceNow applications, comprising a presentation layer (Service Catalog / Service Portal), a process automation layer (Flow Designer), and a data layer (custom tables and the platform's approval engine). The diagram below summarizes the high-level flow of a network request through the system.

Step 1: User submits a 'Network Request' from the Service Catalog.

Step 2: Catalog variables capture request details (connection type, address, device information).

Step 3: Flow Designer trigger fires on creation of the Requested Item (RITM).

Step 4: Flow retrieves catalog variables and creates a record in the custom Network Request table.

Step 5: Flow initiates an approval task routed to the designated approver via Sysapprovals.

Step 6: Based on the approval outcome, flow logic branches to either fulfillment or rejection.

Step 7: Email notifications are sent to the requester and approver at each key stage.

Step 8: On fulfillment, the request record and RITM are updated and closed.

This architecture ensures a clean separation between data capture, business logic, and communication, making the application easier to maintain and extend over time.

6. ServiceNow Developer Environment Setup

Before implementation, a ServiceNow developer environment was provisioned. The following steps describe the setup process used for this project.

1. Created a free account on the ServiceNow Developer Portal (developer.servicenow.com/dev.do).
2. Verified the registered email address and logged in to access the Developer Portal home page.
3. Requested a Personal Developer Instance (PDI), a dedicated cloud-hosted ServiceNow instance for development and testing.
4. Waited for instance provisioning to complete and noted the instance URL and admin credentials.
5. Logged into the PDI as an administrator and explored the Application Navigator.
6. Used Creator Studio / Studio for low-code and no-code application building throughout the project.

7. Project Implementation in ServiceNow

7.1 Service Catalog Creation

The first step of implementation was to create a custom catalog item that end users could use to raise network requests. This was done by navigating to **Service Catalog** → **Maintain Items** → **New** and creating an item named '**Network Request**'.

A set of catalog variables was then configured to capture the necessary details for each request. These variables dynamically adapt the form depending on the type of request being raised:

- **New connection or relocation** – a Choice variable allowing the user to specify whether the request is for a brand-new network connection or relocation of an existing one.
- **Relocated address** – a String variable capturing the new address, shown only when relocation is selected.
- **Device type** – a Choice variable (Laptop / Mobile / Others) identifying the device requiring network access.
- **Device details** – a String variable capturing additional information such as device model or asset tag.

7.2 Table Creation

A custom database table was created to persist network request data independently of the catalog request record, enabling easier reporting and downstream processing. This was configured via **System Definition** → **Tables** → **New**, where the table name, label, and schema were defined. Relevant fields — such as request type, requester, device details, and status — were added to the table using the Columns related list.

7.3 Request Approvals

To enable native ServiceNow approval functionality on the custom table, a relationship was established with the platform's standard Sysapprovals (Approval) table. This was done via **System Definition** → **Relationships** → **New**, after which the related list was added to the custom table's form using Form Designer. This allows approval records to be viewed and managed directly from the network request record.

7.4 Flow Designer Implementation

The core automation logic was implemented using Flow Designer. A new flow named '**Network Request**' was created with the following configuration:

Trigger: Service Catalog → Requested Item (fires when a new Network Request RITM is created).

Flow Actions:

- **Get Catalog Variables** – retrieves the values submitted by the requester on the catalog form.
- **Create Record** – creates a corresponding entry in the custom Network Request database table using the retrieved variable values.
- **Ask for Approval** – generates an approval task and routes it to the designated approver.
- **Flow Logic (If Conditions)** – branches the flow based on the approval outcome (approved or rejected).

- **Send Email** – notifies the requester and approver of status changes at each key milestone.
- **Update Record** – updates the request record and the RITM with the final status once the request has been fulfilled or rejected.

This combination of trigger, data actions, approval logic, and notifications allows the entire network request lifecycle to run automatically once a user submits a request through the catalog, with human involvement limited to the approval decision and any manual fulfillment steps.

8. Testing and Validation

After implementation, the solution was tested end-to-end on the Personal Developer Instance to validate correct functioning of each component. The following test scenarios were executed:

#	Test Scenario	Expected Result	Outcome
TC-01	Access 'Network Request' item from Service Catalog	Catalog item is visible in the portal and the form loads with all configured variables	Passed
TC-02	Submit a new connection request (Choice: New Connection, Device: Laptop)	RITM record is created in ServiceNow; Flow Designer trigger fires on RITM creation	Passed
TC-03	Submit a relocation request with relocated address entered	Relocated address variable value is correctly captured and stored against the RITM	Passed
TC-04	Verify Get Catalog Variables action in Flow Designer	All submitted variable values (request type, address, device type, device details) are retrieved accurately by the flow	Passed
TC-05	Verify Create Record action writes to custom Network Request table	A new record appears in the custom table with field values matching the submitted catalog variables	Passed
TC-06	Verify Ask for Approval action generates an approval task	An approval record is created in the Sysapprovals table and the approver receives an approval task notification	Passed
TC-07	Approver approves the request from the Network Request record	Flow If-condition evaluates to Approved; fulfillment email is sent to requester; RITM and record updated to Closed Complete	Passed
TC-08	Approver rejects the request from the Network Request record	Flow If-condition evaluates to Rejected; rejection email is sent to requester; RITM and record updated to Closed Incomplete	Passed

All test scenarios completed successfully, confirming that catalog data capture, record creation, approval routing, conditional flow logic, and email notifications all function as designed.

9. Screenshots of Output

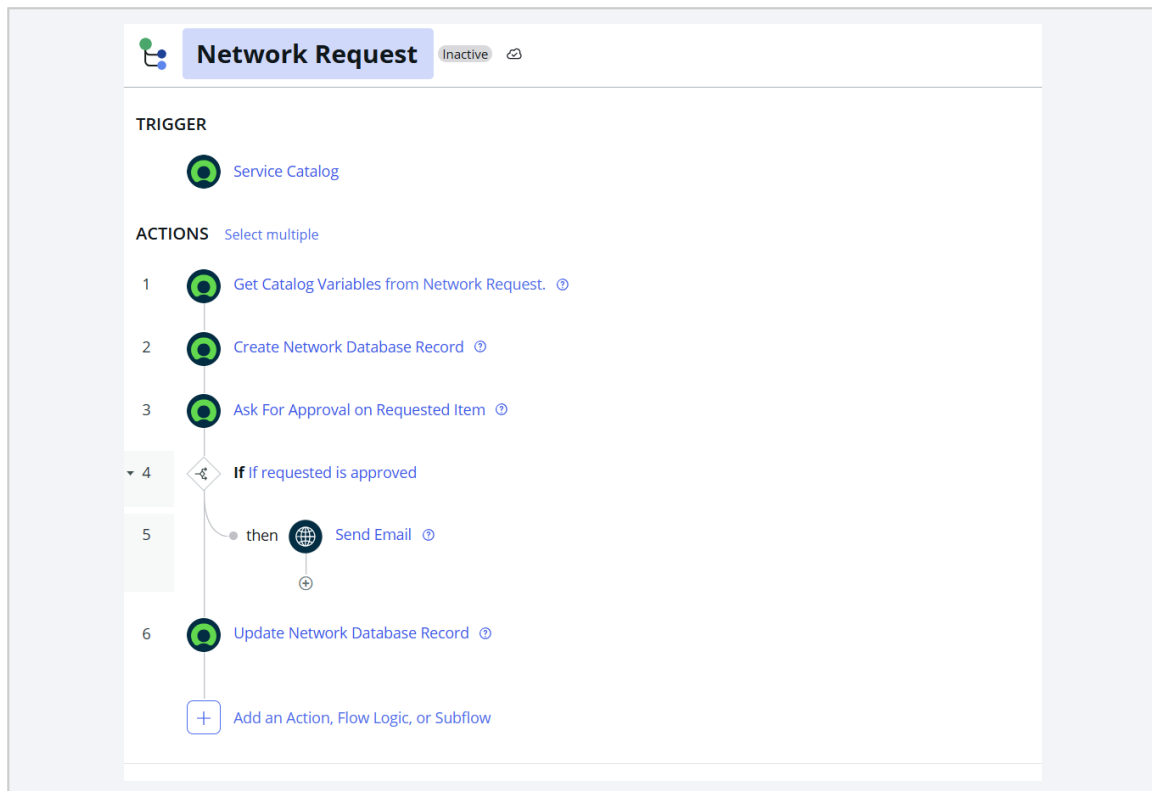
The following screenshots were captured directly from the ServiceNow Personal Developer Instance during implementation and testing. They demonstrate the key stages of the automated Network Request Management workflow.

Figure 1 — Network Request Management: Service Catalog Form

The screenshot displays the 'Network Request Management' form within the ServiceNow interface. The breadcrumb trail at the top reads 'Service Catalog > Networks > Network Request.'. A search bar is located in the top right corner. The form is divided into two main sections. The left section, titled 'Network request Management', contains several fields: a dropdown menu for 'Is this a New connection or Relocation?' with 'New' selected; a text field for 'Opened on behalf of' with a search icon; text fields for 'Email Id', 'User name', and 'Phone Number'; a 'Proof of Document' section with a 'Click to add...' link; a dropdown for 'Types of devices' with 'Laptop' selected; a text field for 'Please provide address here'; and a text field for 'If anything else, please specify'. The right section, titled 'Order this Item', includes a 'Quantity' dropdown set to '1', a 'Delivery time' of '2 Days', an 'Order Now' button, an 'Add to Cart' button, and a 'Shopping Cart' section showing 'Empty'. Mandatory fields are indicated by a red asterisk.

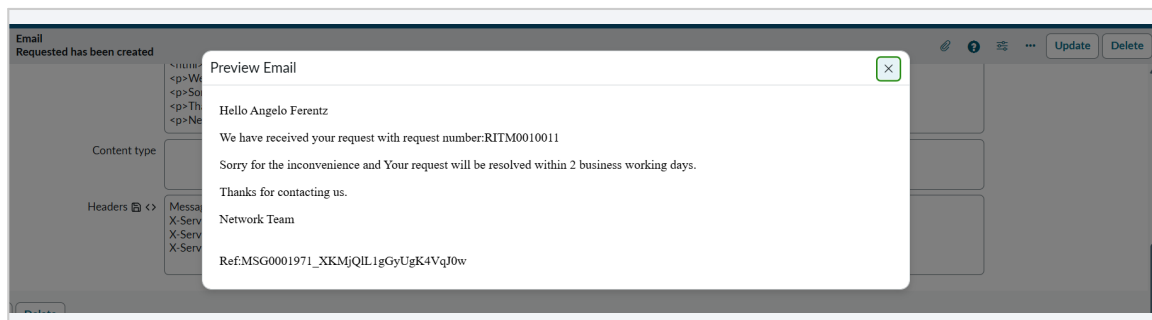
The 'Network Request Management' catalog item as presented to the end user. The form captures required fields including connection type (New / Relocation), requester details (Opened on behalf of, Email ID, Username, Phone Number), Proof of Document, device type, address, and any additional notes. Mandatory fields are marked with a red asterisk.

Figure 2 — Flow Designer: Network Request Workflow



The Flow Designer canvas showing the complete 'Network Request' automation flow. The Service Catalog trigger fires on item submission, followed by six sequential actions: (1) Get Catalog Variables, (2) Create Network Database Record, (3) Ask For Approval on Requested Item, (4) If — request is approved, (5) Send Email notification, and (6) Update Network Database Record.

Figure 3 — Automated Email Notification Preview



Preview of the automated email generated by the Send Email action in the flow. The message confirms receipt of the request (RITM0010011) to the requester Angelo Ferentz, states the expected resolution time of 2 business working days, and is signed by the Network Team.

Figure 4 — Flow Execution Details: Test Run Completed

EXECUTION DETAILS Network Request					
Show Action Details					
FLOW STATISTICS		Run as: System Administrator	Open flow logs	State	Start time
TRIGGER					
Catalog Item Requested					
ACTIONS					
1	Get Catalog Variables from Network Request.	Core Action	Completed	2026-06-30 08:57:27	22ms
2	Create Record	Core Action	Completed	2026-06-30 08:57:27	10ms
3	Ask For Approval	Core Action	Completed	2026-06-30 08:57:27	1138ms
4	If If requested is approved	Flow Logic	Evaluated - True	2026-06-30 09:01:05	35ms
5	Send Email		Completed	2026-06-30 09:01:05	34ms
6	Update Record	Core Action	Completed	2026-06-30 09:01:05	9ms
ERROR HANDLER					

The Execution Details screen showing a successful test run of the 'Network Request' flow completed on 2026-06-30. All six actions — Get Catalog Variables (22 ms), Create Record (10 ms), Ask For Approval (1138 ms), If condition evaluated True (35 ms), Send Email (34 ms), and Update Record (9 ms) — completed with status 'Completed'. Total execution time was 1218 ms.

10. Results and Benefits

The implementation successfully demonstrates a fully automated, end-to-end network request management process on the ServiceNow platform. The key results and benefits observed are summarized below.

- **Reduced turnaround time:** Requests move automatically from submission to approval and fulfillment without manual handoffs, significantly shortening processing time.
- **Improved SLA compliance:** Structured workflows and timely notifications reduce the risk of requests being overlooked or delayed.
- **Greater transparency:** Requesters can track the live status of their request at any time through the self-service portal.
- **Consistent approvals:** All requests follow the same standardized approval path, reducing inconsistency and manual errors.
- **Reduced IT workload:** Automation of routing, notifications, and record creation frees IT staff to focus on actual fulfillment rather than administrative tracking.
- **Auditability:** Every request, approval, and status change is logged within ServiceNow, providing a complete audit trail.

11. Challenges and Learnings

During the course of this project, a number of practical challenges were encountered and resolved, contributing to a deeper understanding of ServiceNow application development.

- **Dynamic catalog variables:** Configuring variable visibility so that fields such as 'Relocated address' only appear when relevant required careful use of variable set conditions.
- **Linking custom tables to approvals:** Establishing the relationship with the Sysapprovals table required understanding ServiceNow's underlying approval data model rather than relying on a simple form field.
- **Flow Designer logic branching:** Designing clear If-condition branches for approved versus rejected outcomes, while keeping the flow easy to read and maintain, required several iterations.
- **Testing across roles:** Validating the approval flow required switching between requester and approver impersonation within the PDI to confirm correct behavior end-to-end.

12. Future Enhancements

While the current implementation fulfills the core objectives of the project, several enhancements could further improve the solution in a production environment:

- Integration with real network orchestration tools (e.g., Ansible, Cisco DNA Center) to fully automate fulfillment of standard requests such as VLAN provisioning.
- Addition of a custom Performance Analytics dashboard for deeper SLA and trend analysis.
- Implementation of multi-level approvals based on request cost, criticality, or department.
- Mobile-optimized Service Portal widgets for raising and tracking requests on the go.
- Use of Virtual Agent / chatbot capabilities to allow users to raise and check on requests conversationally.

13. Conclusion

The Automated Network Request Management project successfully demonstrates how ServiceNow's Service Catalog, custom tables, approval framework, and Flow Designer can be combined to deliver a streamlined, reliable, and user-friendly solution for handling network-related service requests.

By automating approvals, routing, and notifications, the solution improves SLA compliance and user satisfaction while substantially reducing repetitive manual workload for IT teams. The architecture is designed to be extensible, allowing future integration with network automation tools to support fully hands-off fulfillment of standard requests. Overall, this project illustrates the practical value of low-code platform automation in supporting efficient, high-quality, and responsive network services within an enterprise IT environment.

14. References

- ServiceNow Developer Portal — <https://developer.servicenow.com/dev.do>
- ServiceNow Product Documentation — Service Catalog and Flow Designer modules.
- SmartInternz ServiceNow Certified Application Developer (CAD) course materials.
- ServiceNow Community — best practices for catalog variable sets and approval workflows.